



Trabajo Práctico
Estructura de Datos

Fecha de entrega: 03/06/2026

Josue Leonel Gatica Odató 178921

Josuegaticaodato@gmail.com





Aclaración importante sobre colecciones:

Para un correcto manejo de las colecciones en cada ejercicio, los archivos .py tomando las colecciones por defecto en una carpeta llamada “Colecciones”, ubicada a la par con todas las carpetas de los ejercicios.

Nombre	Fecha de modificación	Tipo	Tamaño
coleccion_debug	5/6/2026 01:53	Carpeta de archivos	
dump10k	4/6/2026 14:51	Carpeta de archivos	
test	3/6/2026 03:24	Carpeta de archivos	
wiki-large	3/6/2026 15:56	Carpeta de archivos	
wiki-small	3/6/2026 15:55	Carpeta de archivos	
dump10k.tar.gz	4/6/2026 14:26	Carpeta de archiv...	61.470 KB
queriesDump10K.txt.tar.gz	4/6/2026 14:25	Carpeta de archiv...	106 KB
wiki-large.tar.gz	3/6/2026 01:36	Carpeta de archiv...	536.005 KB
wiki-small.tar.gz	3/6/2026 00:57	Carpeta de archiv...	26.463 KB

En ejercicios donde no se indica el pasaje del directorio donde se encuentran los documentos como parámetro, este toma por defecto la carpeta indicada con el nombre de la colección que se solicitó analizar para ese ejercicio.

Además, para un correcto testeo de los algoritmos, se utilizó una colección llamada “coleccion_debug” que contiene un corpus de ejemplo visto en clase, con tres documentos y cuatro términos (casa, perro, gato, auto).

Por último, los dos índices utilizados para el trabajo se encuentran en la raíz del proyecto:

- Index-large: Colección snapshot de Wikipedia
- Index-small: Colección snapshot de Wikipedia para debug

Herramienta de IA

El modelo de lenguaje de inteligencia artificial utilizado para el trabajo fue ChatGPT



1) Codifique un script que indexe una colección que requiera el volcado parcial a disco (asumiendo que existe un límite de memoria) implementando el método BSBI visto en clase. Para esto su script debe recibir un parámetro n que indica cada cuántos documentos se debe hacer el volcado a disco. Al finalizar, debe unir (merge) los índices parciales. Para las pruebas use la colección snapshot de Wikipedia y varios valores de n (por ejemplo, $n = 10\%$ del tamaño de la colección). Registre los tiempos de indexación y de merge por separado. Grafique la distribución de tamaños de las posting lists. Calcule el overhead de su índice respecto de la colección. ¿Qué conclusiones se pueden extraer?

1.1) Agregue un script que cargue el vocabulario de la colección en memoria, permita recuperar una posting completa de un término y la muestre por pantalla. Para cada documento retornado se deberá mostrar el nombre, el docID asignado durante la creación del índice y la frecuencia en el siguiente formato:

- DocName:docID:Frecuencia

Utilice la colección de debug para calibrar su script y verificar que la salida sea correcta.

Resultados obtenidos de la colección Wiki-Small

Ejecutando el algoritmo BSBI para la colección Wiki-Small, se obtuvieron los siguientes resultados

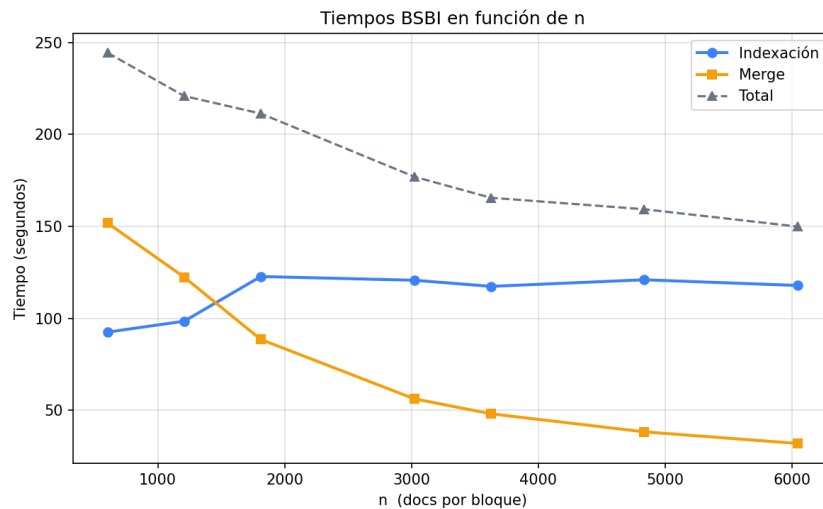
Colección	3042 Mb
Índice binario	298 Mb
Vocabulario (Pickle)	103 Mb
Total índice	401 Mb
Overhead (Índice/Colección)	13.2%
Términos	3511498
Documentos	121818

Además, se llevó a cabo la construcción del índice para diferentes valores de n , que va desde el 10% del tamaño a la colección hasta el total de la misma. En este caso, se utilizó la colección de Debug.

Ejecutadas las corridas, se obtuvieron los siguientes resultados:

Tamaño de N con respecto al tiempo de ejecución

Este grafico muestra los tiempos correspondientes a la indexación, merge y total entre ambos para los diferentes valores de n que se utilizaron para la construcción del índice.



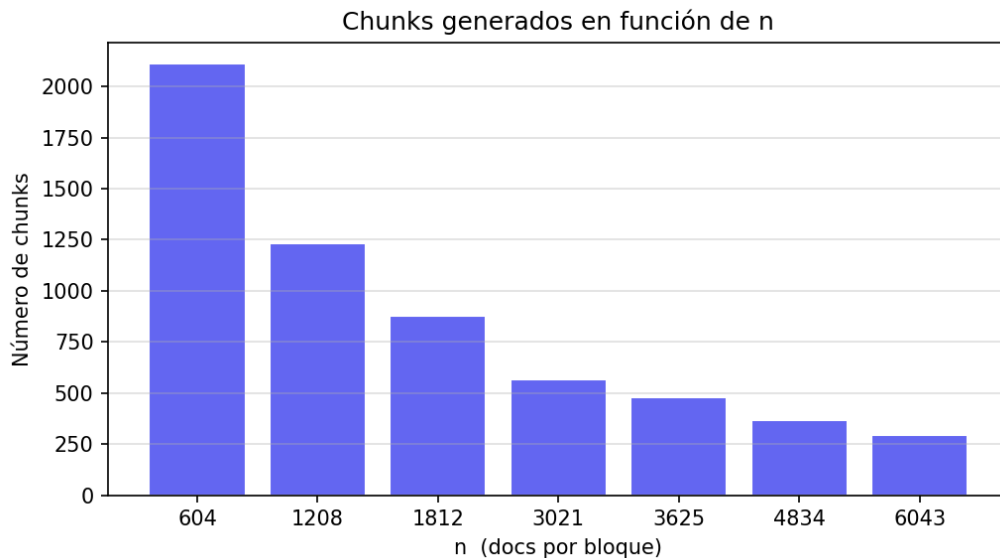
En consideración al total, consume más tiempo tener un n menor para la construcción del índice, debido a que tengo una mayor cantidad de chunks, siendo necesario realizar un volcado a disco constante de la tupla en memoria, aumento este el tiempo de procesamiento.

El merge resulta ser una operación costosa para bajos valores de n. El aumento de este parámetro hace que los tiempo sean muchos menores, ya que baja la cantidad de chunks, implicando una menor cantidad de operaciones lectura/escritura para construir el índice.

La indexación, en valores de n que ronda de 2000 a 6000 (30% a 100% del tamaño de la colección aproximadamente) se mantiene relativamente estable en tiempo, con un ligero aumento inicial que luego se estabiliza. Esto se explica ya que la colección a procesar siempre es la misma, lo único que varía es el tamaño de la escritura a disco que realizo para los chunks.

Como conclusión general, puede decirse que aumentar la cantidad del parámetro n es relevante para el tiempo de procesamiento del índice. Es necesario maximizar el valor de n hasta el límite de la memoria para aumentar el rendimiento general del algoritmo.

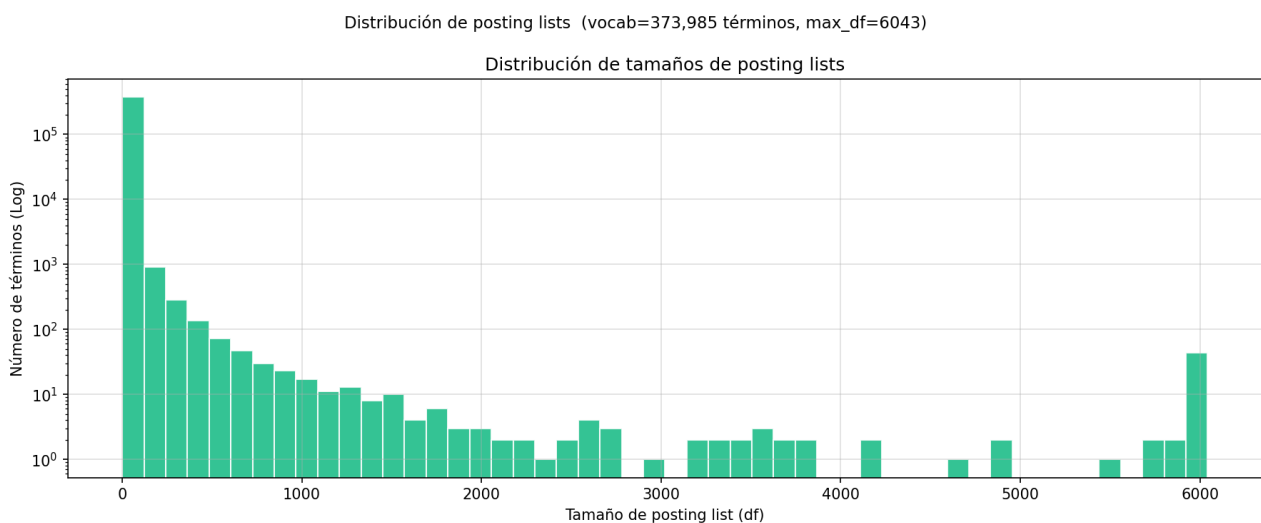
Cantidad de chunks con respecto a N



Como era de esperarse, existe una relación inversamente proporcional, donde al aumentar la capacidad de documentos por bloque, disminuye la cantidad de chunks necesarias para procesar la colección.

Es de interés destacar que, del 10% al 20%, la cantidad de chunks se reduce aproximadamente a la mitad. Luego este se reduce un tercio de este y así sucesivamente. Cada vez es menor la cantidad de chunks, pero llega a un valor de tendencia donde varía cada vez menos, que sería 260/250 aproximadamente en este caso.

Distribución de postings lists



En base al gráfico, podemos decir que la ley de Zipf se cumple parcialmente. Alrededor de diez mil término del vocabulario (1/3 del total) tiene un document frequency cercano a cero, indicando que aparecen en muy pocos documentos. Además, existen una gran cantidad de



términos que están presentes en los seis mil documentos utilizados para la indexación; estos pueden tratarse de stopwords o palabras que siempre usa Wikipedia en sus documentos (wiki, artículo, etc.).

Por último, puede verse como las postings tienen una media aproximada de 10 a 100 documentos, haciendo que sea rápido para el procesamiento de consultas.

Recuperación de la posting

Construido el índice, se elaboró un script (EJ1-Posting.py) para la recuperación de la posting completa correspondiente a un término.

```
python TP4/EJ1/EJ1-Posting.py spirit --index-dir TP4/index-small --index-name index-small
```

Consultando por el término “spirit”, se obtuvo la siguiente posting:

```
Posting list del término 'spirit' (df=58):  
Formato: DocName:docID:Frecuencia  
-----  
0_(number):2:1  
A_Short_Organum_for_the_Theatre_8cc2:165:1  
Akhlut:255:2  
Apocalypse_Now_a6a5:455:1  
Arran_Single_Malt_208e:492:2  
....
```

Utilización de IA:

Fue necesaria la utilización de la inteligencia artificial para:

- Un correcto algoritmo para la lectura de los documentos de la colección
- Armado del algoritmo de BSBI, tanto la indexación y merge
- Creación de gráficos para el análisis posterior.



2) Codifique un script que implemente la estrategia TAAT vista en clase y sobre el índice creado en el ejercicio 1 permita realizar operaciones sobre conjuntos para buscar por dos o tres términos utilizando los operadores AND, OR y NOT.

Su script debe permitir el procesamiento de consultas del tipo:

- ((t1 AND t2) OR t3)
- ((t1 AND NOT t2) OR NOT T3))

Como salida su script debe retornar el nombre y el docID de los documentos que satisfacen la consulta.

Utilizando la estrategia TAAT y el modelo booleano, realizamos la siguientes consultas sobre el índice ejecutando el siguiente comando:

```
python TP4/EJ2/EJ2.py "((spirit AND encyclopedia) AND article)" --index-dir TP4/index-small --index-name index-small
```

En este caso se probó sobre la colección de debug. Si se quisiera probar sobre la principal, se debería indicar “index-large” en la dirección y nombre del índice.

Ejecutando la consulta, se obtuvieron los siguientes documentos que satisfacen la consulta:

```
[TAAT] Vocabulario: 373985 términos | Colección: 6043 documentos
Consulta : ((spirit AND encyclopedia) AND article)
Resultados: 33 documento(s)
-----
DocName                                docID
-----
@_(number)                             2
Akhlut                                  255
Apocalypse_Now_a6a5                     455
Arran_Single_Malt_208e                   492
....
```

Otro ejemplo

```
[TAAT] Vocabulario: 373985 términos | Colección: 6043 documentos
Consulta : ((states AND democracy) AND NOT article))
Resultados: 2 documento(s)
-----
DocName                                docID
-----
Battle_of_Crannon_e40d                   647
Central_European_Free_Trade_Agreement_1afb 1050
```

Utilización de IA:

Fue necesaria la utilización de la inteligencia artificial para el armado de los operadores binarios y la evaluación de las consultas



3) Utilizando el código e índice anteriores ejecute corridas con el siguiente subset de queries (filtre solo los de 2 y 3 términos que estén en el vocabulario de su colección) y mida el tiempo de ejecución en cada caso. Para ello, utilice los siguientes patrones booleanos:

A. Queries $|q| = 2$

- $t1 \text{ AND } t2$
- $t1 \text{ OR } t2$
- $t1 \text{ NOT } t2$

B. Queries $|q| = 3$

- $t1 \text{ AND } t2 \text{ AND } t3$
- $(t1 \text{ OR } t2) \text{ NOT } t3$
- $(t1 \text{ AND } t2) \text{ OR } t3$

¿Puede relacionar los tiempos de ejecución con los tamaños de las listas? (pruebe con el índice en disco o cargándolo completamente en memoria antes). ¿Qué conclusiones se pueden extraer?

Resultados por query

Mediante el código e índice del ejercicio anterior, se ejecutaron corridas sobre un subset de queries, realizando una correcta tokenizacion y eliminando stopwords, utilizando solamente aquellas queries que poseen dos a tres términos. Además, se probó el uso del índice en disco o cargando completamente en memoria.

Los resultados fueron los siguientes:

Queries $|q| = 2$

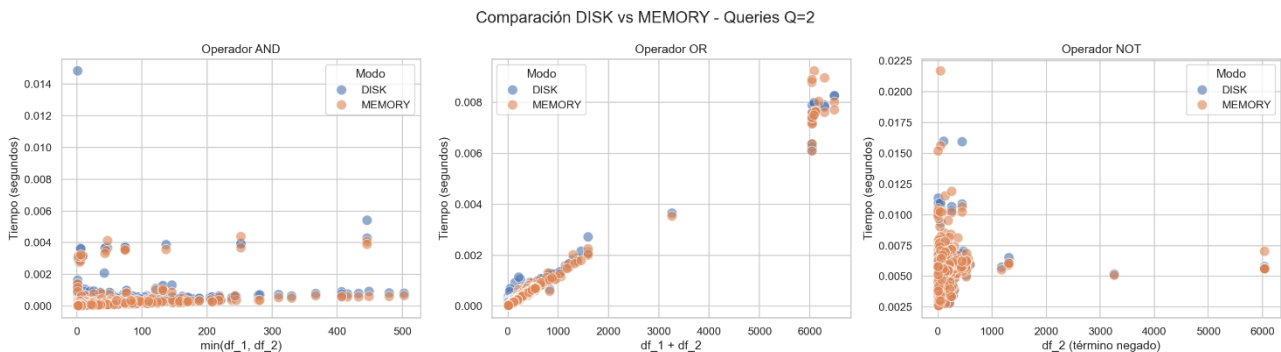
En Disco	AND	OR	NOT
Cant. Queries	1657	1657	1657
Tiempo promedio	0.000354s	0.000413s	0.005752s
Tiempo mínimo	0.000150s	0.000131s	0.002716s
Tiempo máximo	0.014837s	0.008267s	0.015972s

Puede verse como la operación AND es la menos costosa computacionalmente en cuanto a tiempo, y la operación NOT es la que mas tiempo demora, ya que tiene que revisar aquellas docs en T2 que no están en T1.

En Memoria	AND	OR	NOT
Cant. Queries	1657	1657	1657
Tiempo promedio	0.000140s	0.000289s	0.005746s
Tiempo mínimo	0.000015s	0.000014s	0.002609s

Tiempo máximo	0.004379s	0.009240s	0.021695s
---------------	-----------	-----------	-----------

En memoria, los tiempos fueron muchos mejores en comparación a disco, bajando a la mitad en operaciones AND y OR, pero manteniéndose igual en NOT.



En este gráfico, puede verse la comparación entre disco y memoria en cuanto a tiempo y el tamaño de las posting.

Según la operación que se realice, el tamaño de la posting varia:

- En operaciones AND, el tamaño de la posting que se utiliza para evaluar el tiempo es el termino que tiene la posting más corta.
- En operaciones OR, es necesaria sumar ambos tamaños de posting
- En operaciones NOT, depende todo del tamaño de la posting del término negado.

Es de interés ver como el operador NOT se mantiene en valores de tiempo muy cercanos entre 0.0025 y 0.0125, sin importar que se encuentra en memoria o disco. Este operador resulta ser mas inestable, dependiendo mucho de la posting del término negado.

El operador OR se puede decir que ambas variables están fuertemente correlacionadas linealmente, donde a medida que aumenta el tamaño de las postings, también crece el tiempo. Esto sucede tanto para disco como memoria.

Por último, el operador AND es el que menor tiempo ocupa, ya sea en disco o en memoria, tendiendo a ser una función con una pendiente con muy poca inclinación, indicando que aumentar el tamaño de la posting no aumenta significativamente el tiempo. Esto se justifica por el hecho de que se recorre siempre la posting más corta.

Queries $|q| = 3$

En Disco	AND-AND	OR-AND-NOT	AND-OR
Cant. Queries	936	936	936
Tiempo promedio	0. 000505s	0. 006783s	0. 000653s
Tiempo mínimo	0. 000203s	0. 004109s	0. 000209s

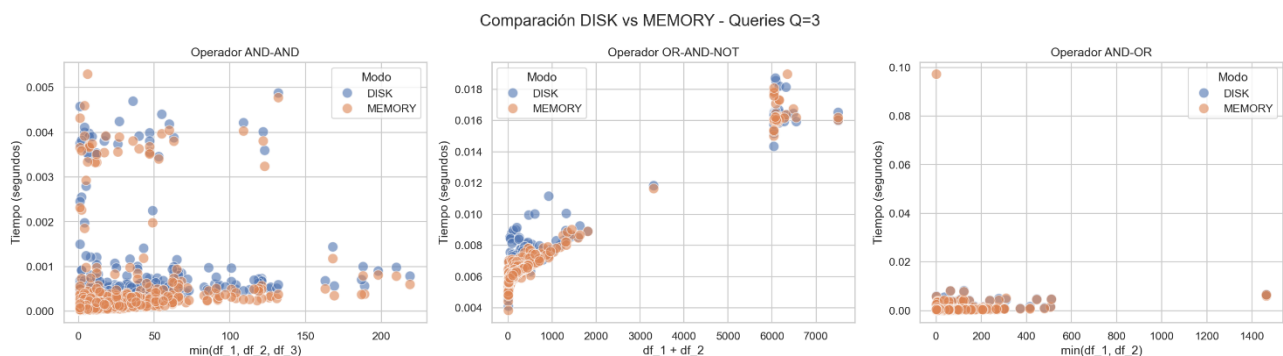
Tiempo máximo	0. 004878s	0. 018705s	0. 008124s
---------------	------------	------------	------------

Como era de suponerse, aumentar la cantidad de queries a operar también implica aumentar tiempos de procesamiento, sin importar que la cantidad de queries se redujo de 1600 a 900 para cada operación.

Realizar tres operaciones binarias (OR-AND-NOT) es lo mas costoso en procesamiento. Una operación AND-AND va a ser más rápida que realizar un AND-OR.

En Memoria	AND-AND	OR-AND-NOT	AND-OR
Cant. Queries	936	936	936
Tiempo promedio	0. 000287s	0. 006645s	0. 000498s
Tiempo mínimo	0. 000018s	0. 003837s	0. 000022s
Tiempo máximo	0. 005297s	0. 018957s	0. 097186s

En memoria, al igual que las queries de dos términos, los tiempos fueron muchos mejores en comparación a disco, haciéndose notar mucho en operaciones AND-AND, donde el tiempo se redujo a la mitad.



Se muestra gráficamente los tres operadores para queries de tres términos, evaluados en cuanto a tiempo.

El doble AND sigue la misma lógica que el operador AND de dos términos, teniendo tiempos muy bajos, tanto en memoria como disco.

Las tres operaciones juntas traen consigo una tendencia, muy similar a la correlación que se menciona con el operador OR. Esta resulta ser la operación mas costosa de las tres.

En el operador AND-OR, se esta mezclando tanto una operación “barata” en términos de procesamiento, con una más costosa, como es el or, haciendo que el T3, a quien se le aplica el or, determine los tiempos.

En conclusión, podemos decir que tener el índice en memoria es ligeramente mas rápido, pero la diferencia no es tanta como para hacerlo siempre. Lo que mas influencia tiene en



estos casos es el tamaño de la posting, dependiendo el operador binario que estemos aplicando; este es el que varía el tiempo de procesamiento realmente.

Utilización de IA:

Fue necesaria la utilización de la inteligencia artificial para la evaluación de queries, tanto de dos y tres términos, además de la elaboración de gráficos para el posterior análisis



4) Codifique un script que implemente la estrategia DAAT vista en clase y sobre el índice creado en el ejercicio 1 permita resolver consultas usando el modelo vectorial utilizando la métrica del coseno como medida de similitud. Dada una consulta su script debe retornar los top-k documentos de score mayor.

Su script debe mostrar como salida el nombre, el docID y el score (ordenado por score) con el siguiente formato

●DocName:docID:Score

Utilizando la estrategia DAAT y el modelo vectorial, realizamos las siguiente consultas sobre el índice, indicando un k cantidad de documentos a devolver para el top, ejecutando el siguiente script:

```
python TP4/EJ4/EJ4.py "spirit encyclopedia article" --index-dir TP4/index-small --index-name index-small -k 5
```

En este caso se probó sobre la colección de debug. Si se quisiera probar sobre la principal, se debería indicar "index-large" en la dirección y nombre del índice.

Ejecutando la consulta, se obtuvieron los siguientes documentos con su ranking, respetando el formato solicitado:

```
Bezoardicum:732:0.225582  
Sukra:5142:0.142188  
Akhlut:255:0.106989  
Toyol:5502:0.096942  
Dinh:1471:0.094783
```

Otro ejemplo

```
python TP4/EJ4/EJ4.py "states democracy article"--index-dir TP4/index-small --index-name index-small -k 5  
  
List_of_Mexican_states_by_Human_Development_Index_0734:3213:0.097478  
List_of_sovereign_states_in_1941:3294:0.080272  
Observability:3992:0.075177  
September_28,_2003:4823:0.070884  
List_of_Nigerian_traditional_states_9277:3218:0.067958
```

Utilización de IA:

Fue necesaria la utilización de la inteligencia artificial para la construcción del algoritmo de DAAT junto con el modelo vectorial, para el cálculo de la norma de los documentos junto con los TF-IDF.



5) Agregue skip lists a su índice del ejercicio 1 y ejecute un conjunto de consultas AND sobre el índice original y luego usando los punteros. Compare los tiempos de ejecución con los del ejercicio 3.

5.1) Agregue un script que permita recuperar las skips list para un término dado. En este caso la salida deberá ser una lista de docName:docID ordenada por docName.

Utilice la colección para debugging para calibrar su script.

Consultas AND sobre skips list

Ejecutando el siguiente script:

```
python TP4/EJ5/EJ5.py "spirit" "encyclopedia" --index-dir TP4/index-small --index-name index-small
```

Se construyo el siguiente algoritmo:

1. Obtener las postings de ambos términos
2. Evaluar cual de las dos postings es la más corta
 - a. La posting más corta será de pivot
 - b. La posting más larga se le aplicara una skip lists
3. Comparar la skip lists con la posting pivot
 - a. El pivot obtiene un docID
 - b. Este numero se compara con el max_docID que tiene el bloque de la skip
 - i. Si es mayor, skipeamos el bloque
 - ii. Si es menor, quiere decir que el valor puede estar dentro. Hacemos una búsqueda secuencial
 - c. Cada vez que se matchea entre la skip y la posting, se guarda el docID resultado

De esta forma, se obtuvieron los siguientes resultados:



```
Término 'spirit' (corto): 58 docs
Término 'encyclopedia' (largo): 6043 docs

-----
Búsqueda usando Skip List:
-----

Comparaciones skip list: 2400
Comparaciones secuencial: 2219
Total comparaciones: 4619
Tiempo: 0.015513800026383251s

=====
Resultado de spirit AND encyclopedia:
=====
DocIDs encontrados: [2, 165, 255, 455, 492, 581, 726, 732, 852, 1127, 1239, 1471, 1590,
1597, 1670, 1782, 1878, 2009, 2053, 2055, 2098, 2355, 2378, 2379, 2380, 2690, 2819, 2895,
3196, 3214, 3421, 3462, 3528, 3783, 3930, 4062, 4195, 4197, 4220, 4248, 4394, 4736, 4775,
4809, 4943, 4984, 5063, 5142, 5332, 5351, 5371, 5420, 5456, 5502, 5563, 5722, 5840, 5969]

Documentos (cantidad: 58)
```

Ejecutando la misma consulta sobre el ejercicio dos, se obtienen exactamente los mismos documentos como resultado, y un tiempo muy similar, a pesar de que toma la búsqueda fue realizada de forma secuencial.

```
python TP4/EJ2/EJ2.py "(spirit AND encyclopedia)" --index-dir TP4/index-small --index-name
index-small

Consulta : (spirit AND encyclopedia)
Resultados: 58 documento(s)
Tiempo: 0.015104199992492795 s

-----
DocName                                docID
-----
@_(number)                             2
A_Short_Organum_for_the_Theatre_8cc2    165
Akhlut                                  255
Apocalypse_Now_a6a5                     455
...
```

Recuperación de skips list

Consultando a la colección indicada como parámetro, es posible recuperar la skips list de un termino de la siguiente manera:

```
python TP4/EJ5/EJ5-1.py "encyclopedia" --index-dir TP4/index-small --index-name index-
small
```

En este caso se probó sobre la colección de debug. Si se quisiera probar sobre la principal, se debería indicar "index-large" en la dirección y nombre del índice.

El resultado de la recuperación fue una lista compuesta por el nombre del documento junto con el docID.



En mi algoritmo, siempre el docID va a ser el valor máximo del bloque de la posting. Esta decisión fue tomada para el procesamiento de consultas entre dos términos, descartando el bloque entero en el caso de que la posting lists más corta sea mayor a ese máximo.

```
1993-94_in_Dutch_football_6733:77
A_Bridge_Too_Far_(film)_c424:154
Agency_for_Public_Management_and_eGovernment_7003:231
Alfonso_I_90d2:308
Amphoe_Sai_Noi_3eb7:385
AQA_Anthology_32e1:462
Attingal:539
...
```

Utilización de IA:

Fue necesaria la utilización de la inteligencia artificial para una correcta construcción de la skips list y su evaluación junto con la posting, para la obtención de los documentos relevantes de esa query.

6) Sobre la colección Dump10k escriba un programa que realice una evaluación TAAT y otro usando DAAT. Compare los tiempos de ejecución para un conjunto de queries dados. Separe su análisis por longitud de queries y de posting lists.

Resultados obtenidos de la colección Dump10k

Mediante el procesamiento de la colección Dump10k, realizamos la evaluación de los algoritmos de recuperación TAAT y DAAT, utilizando queries que varían de uno a cuatro términos. Los resultados fueron los siguientes:

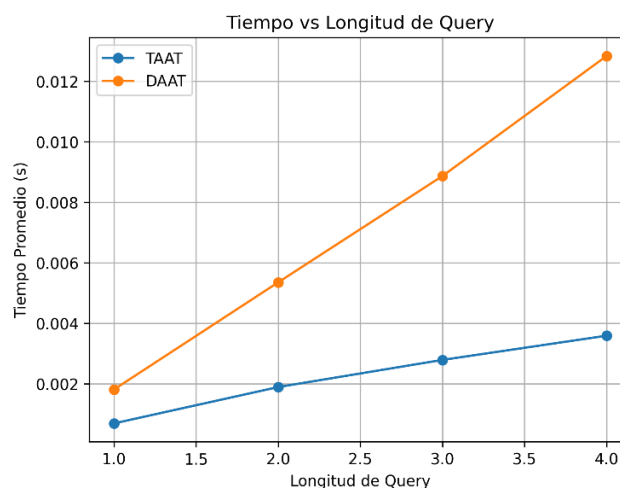
Cant. Terminos	Longitud	AVG Tamaño Posting	TAAT	DAAT
1	2500	1623	0.000779s	0.001882s
2	2500	4143	0.001967s	0.005615s
3	2500	6040	0.002937s	0.008962s
4	2500	7759	0.003796s	0.013174s

El tiempo de los algoritmos esta expresado en promedio

Se realiza la evaluación con un total de diez mil queries, donde estas están divididas exactamente en 2500 queries para cada longitud: 1,2, 3 y 4 términos.

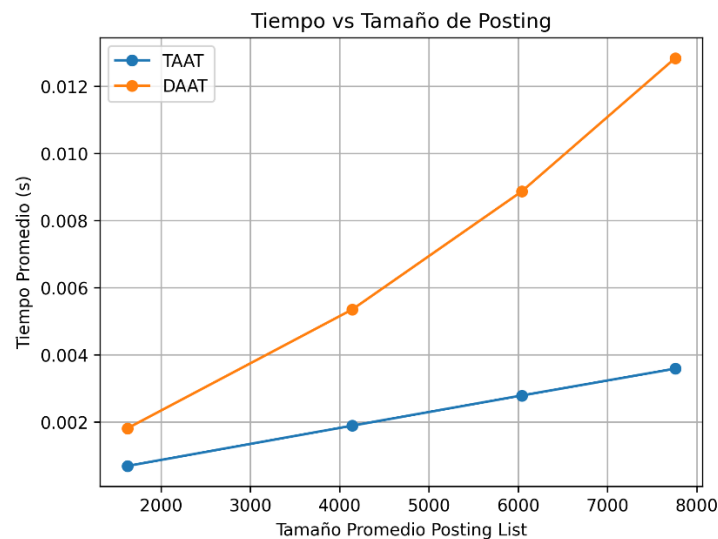
Para una mejor comprensión de los resultados, se construyeron los siguientes gráficos:

Tiempo vs Longitud de Query



Podemos ver cómo, a medida que agregamos mas términos a la query, el tiempo promedio entre ambos algoritmos aumenta, pero es mucho mas leve en TAAT, que tiene una variación de un segundo entre términos; mientras que DAAT escala de forma mucho peor ante el aumento de términos, teniendo una pendiente más empinada, tardando más tiempo. La justificación es la coordinación de punteros que tiene que realizar el algoritmo, además de mantener siempre actualizado el documento mínimo.

Tiempo vs Tamaño de Posting



Al igual que el gráfico anterior, la pendiente de DAAT aumenta más pronunciadamente ante el aumento del tamaño promedio de la posting, debido al procesamiento de documentos en paralelo con cursores que debe realizar el algoritmo para obtener siempre el resultado.

En este ejercicio, el mejor algoritmo para el procesamiento de la colección fue TAAT en términos de velocidad, ya que lo que importo en esta evaluación es la velocidad en cuanto a tamaño de queries y tamaño de posting, sin garantizar un top de documentos relevantes para la consulta.

Utilización de IA:

Fue necesaria la utilización de la inteligencia artificial para la construcción de gráficos que puedan visibilizar los resultados obtenidos del procesamiento.



7) Comprima el índice del ejercicio 1 utilizando Variable-Length Codes (VByte) para los docIDs y Elias-gamma para las frecuencias (almacene docIDs y frecuencias en archivos separados). Calcule tiempos de compresión/descompresión del índice completo y tamaño resultante en cada caso. Realice dos experimentos, con y sin DGaps. Compare los tamaños de los índices resultantes.

7.1) Agregue un script que permita recuperar de disco la posting list de un término dado y la versión comprimida de dicha lista.

Compresión – Con y sin DGaps

Mediante los algoritmos de compresión vistos en clase, se llevo a cabo la compresión del índice, utilizando VByte para los docIDs y Elias para la frecuencia.

Oon un índice de:

- 291.21 MB de tamaño
- 3,511,498 términos
- 121,818 docs

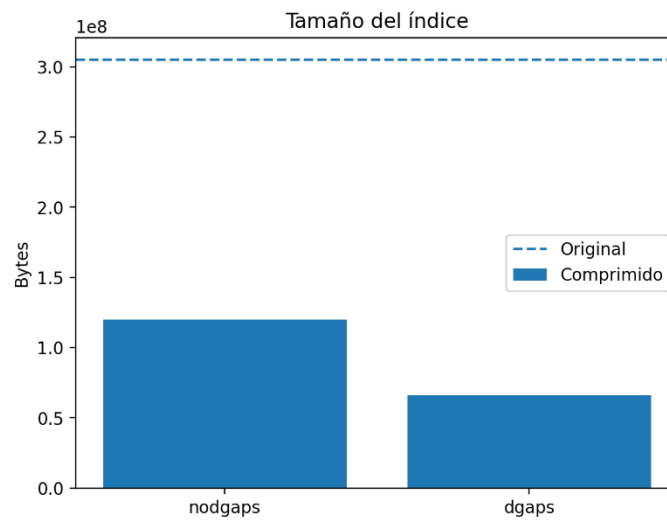
Se obtuvieron los siguientes resultados:

	CON DGaps	SIN DGaps
Tamaño docIDs (VByte)	53.05Mb	104.36Mb
Tamaño freqs (Elias)	10.10Mb	10.10Mb
Total comprimido	63.15Mb	114.45Mb
Tasa de compresión	4.61x	2.54x
Ahorro de espacio	78.3%	60.7%
Tiempo compresión	697.996s	676.498s
Tiempo descompresión	66.898s	83.896s

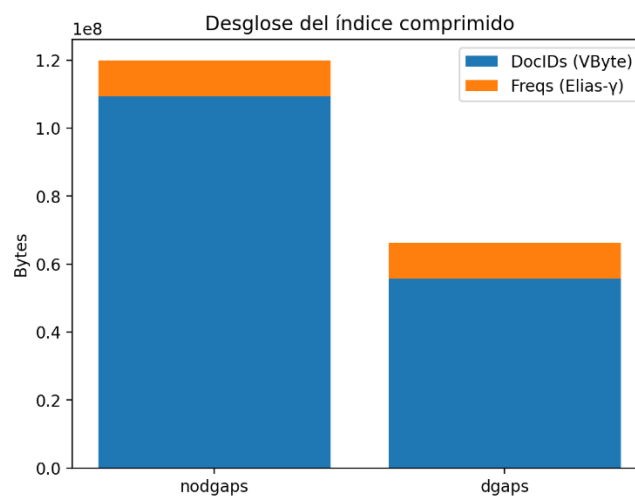
El uso de DGaps trae consigo una mejora significativa en el tamaño del índice, bajando a la mitad el tamaño comprimido que la compresión sin Dgaps, justificado por la transformación de los docIDs en números más pequeños, reduciendo la cantidad de bytes.

El costo adicional de aplicar DGaps es justificado, ya que con 20s más de compresión se ofrece una mejora significativa en espacio para el índice. Además, su descompresión es el 20% más rápida que su competidor, ya que procesa menos bytes.

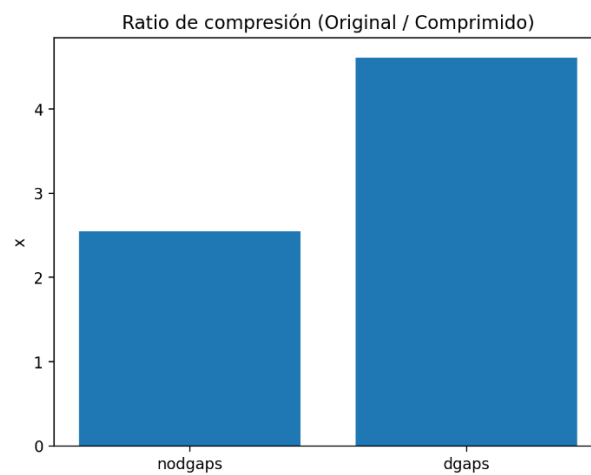
Siguiente al análisis de la tabla, continuamos con los gráficos



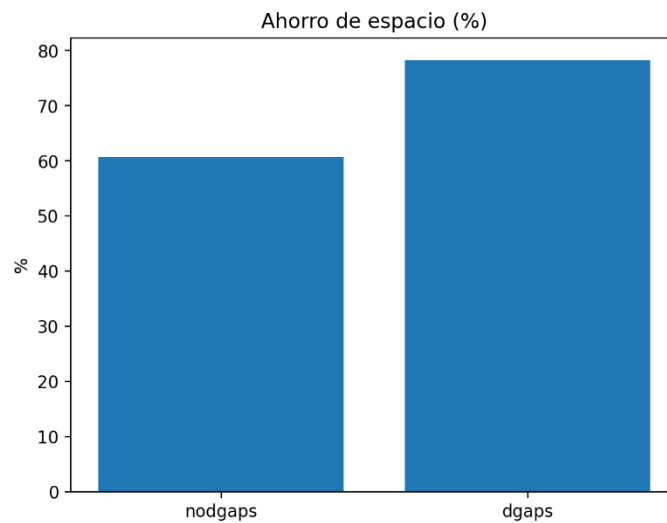
El tamaño del índice comprimido es casi 3 veces menor sin DGaps, y 4 veces menor con DGaps.



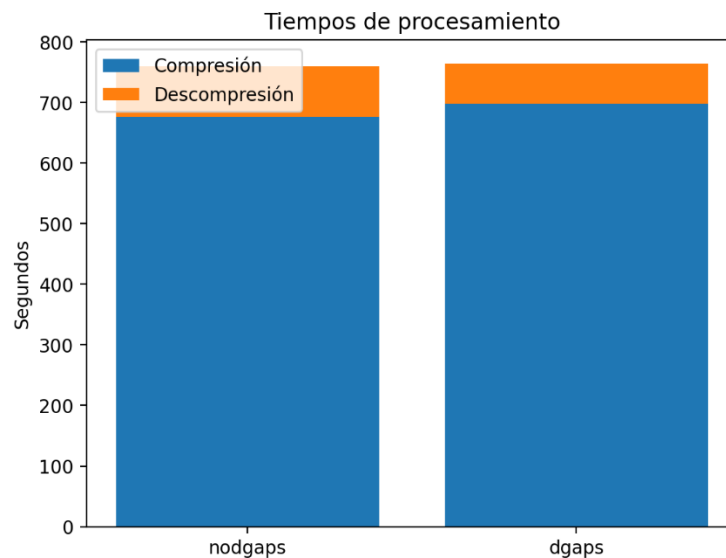
El tamaño del índice comprimido es el doble para noDGraps comparado con DGraps.



El ratio de compresión es del doble para DGaps



DGaps nos ofrece un 20% más de ahorro en el espacio



Y como se vio en la tabla, el tiempo es justificable para usar DGaps, y lo vale por todos los beneficios que ofrece en cuanto a compresión.

En conclusión, decimos que DGaps, junto con VByte y Elias, es altamente efectivo para la compresión de índices invertidos, reduciendo significativamente el tamaño del índice y mejora en los tiempos de descompresión, con un coste justificable en tiempo de procesamiento. Sin embargo, no utilizar DGaps también ofrece una buena variante para compresión, pero en tiempo y espacio es mucho menor.

Recuperación de posting list comprimida

Mediante el siguiente script, es posible recuperar la posting de un termino, ya sea para un indice con o sin DGaps

```
python TP4/EJ7/EJ7-1.py "encyclopedia" --dgaps dgaps
python TP4/EJ7/EJ7-1.py "encyclopedia" --dgaps nodgaps
```



Utilización de IA:

Fue necesaria la utilización de la inteligencia artificial para la elaboración de los algoritmos de compresión, tanto VByte como Elias, y la elaboración de graficos para el posterior análisis.



Bibliografía

- [1] Ricardo Baeza-Yates and Berthier Ribeiro-Neto. Modern Information Retrieval: The Concepts and Technology Behind Search. Addison-Wesley Publishing Company, USA, 2nd edition, 2008.
- [2] Bruce Croft, Donald Metzler, and Trevor Strohman. Search Engines: Information Retrieval in Practice. Addison-Wesley Publishing Company, USA, 1st edition, 2009.
- [3] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. Introduction to Information Retrieval. Cambridge University Press, New York, NY, USA, 2008.
- [4] Stefan Buttcher. and Charles L. A. Clarke and Gordon V. Cormack. Information Retrieval: Implementing and Evaluating Search Engines. MIT Press, 2010.